

Chương 3

Làm việc với file

xem, tạo, sao chép, di chuyển, xóa (an toàn)

Giáo trình Linux cho người mới bắt đầu

Mục lục

Mục tiêu học tập	3
1 Khởi động: vào sân tập (làm trước mọi thứ)	3
2 XEM nội dung file	4
2.1 cat — in TOÀN BỘ file ra màn hình	5
2.2 less — cuộn xem file DÀI một cách thoải mái	5
2.3 head — xem vài dòng ĐẦU	6
2.4 tail — xem vài dòng CUỐI	6
3 TẠO file	7
3.1 touch — tạo file RỖNG (hoặc cập nhật thời gian)	7
3.2 echo + > — tạo file VÀ ghi nội dung (GHI ĐỀ)	7
3.3 echo + >> — NỐI THÊM vào cuối file	8
4 SAO CHÉP file và thư mục — cp	9
4.1 Sao chép một file	9
4.2 Sao chép cả THƯ MỤC — cần -R	9
4.3 cp -i — hỏi trước khi GHI ĐỀ (an toàn)	10
5 DI CHUYỂN và ĐỔI TÊN — mv	11
5.1 Đổi tên	11
5.2 Di chuyển vào thư mục	11
5.3 mv -i — hỏi trước khi GHI ĐỀ	11
6 XÓA file — phần NGUY HIỂM NHẤT, đọc kỹ	12
6.1 rm — xóa một file	12
6.2 rm -i — hỏi xác nhận từng file (an toàn cho người mới)	12
6.3 rmdir — chỉ xóa thư mục RỖNG (an toàn)	13
6.4 rm -r — xóa thư mục VÀ toàn bộ nội dung (đệ quy)	13
6.5 rm -rf và những thảm họa kinh điển — ĐỌC THẬT KỸ	14
7 KÝ TỰ ĐẠI DIỆN (wildcard / glob) — sơ lược nhưng phải biết cảnh giác	15
8 LAB CHÍNH — gõ tay toàn bộ, an toàn 100% (chỉ trong ~/linux-lab/ch3)	17
9 HIỂU LẦM THƯỜNG GẶP (đọc để khỏi “dính bẫy”)	19
10 THẺ GHI NHỚ (flashcards)	20
11 TÓM TẮT 1 TRANG	21
12 BÀI TẬP MỞ RỘNG (stretch) — CÓ đáp án	22
Đáp án — Bài tập mở rộng	22
Đáp án — Tự kiểm tra	24

13 KẾ HOẠCH ÔN NGẮT QUÃNG (spaced repetition)

24

Tôi sẽ sửa bản thảo theo toàn bộ góp ý của hội đồng, ưu tiên các vấn đề critical/major. Dưới đây là toàn bộ Chương 3 đã chỉnh sửa.

Ở Chương 2 bạn đã biết đi lại trong cây thư mục bằng `cd`, `pwd`, `ls`, hiểu đường dẫn, và đã có sẵn **sân tập** `~/linux-lab`. Chương này bạn sẽ học cách **chạm tay** vào chính các tệp tin: mở ra xem, tạo mới, nhân bản, dời chỗ, đổi tên và — phần quan trọng nhất — **xóa một cách an toàn**.

Một câu nói trong nghề: “*Lệnh `cd` sai thì bạn lạc đường; lệnh `rm` sai thì bạn mất dữ liệu.*” Chương này dạy bạn cách **không bao giờ mất dữ liệu vì gõ ẩu**.

Mục tiêu học tập

Học xong chương này, bạn sẽ tự tin:

1. **Xem** nội dung file bằng 4 cách: `cat`, `less`, `head`, `tail` — và biết khi nào dùng cái nào.
2. **Tạo** file bằng `touch`, `echo > file`, `echo >> file` — và phân biệt rạch ròi `>` (ghi đè) với `>>` (nối thêm).
3. **Sao chép** file và thư mục bằng `cp`, `cp -R`, `cp -i`.
4. **Di chuyển và đổi tên** bằng `mv`, `mv -i`.
5. **Xóa an toàn** bằng `rm`, `rm -i`, `rm -r`, `rm -ri`, `rmdir` — và hiểu vì sao `rm` trên Terminal **không** giống “kéo vào Thùng rác”.
6. Hiểu sơ về **ký tự đại diện** `*` và vì sao nó vừa tiện vừa nguy hiểm.
7. Hình thành **bộ thói quen an toàn** mà cả Senior DevOps đều tuân theo (đặc biệt: `pwd` để biết mình ở đâu, `ls` để biết mình sắp xóa gì).

Ghi chú: Thuật ngữ sẽ gặp

(giải thích ngay khi xuất hiện lần đầu trong bài): *lệnh (command)*, *tham số/đối số (argument)*, *tùy chọn/cờ (option/flag)*, *chuyển hướng (redirect)*, *đệ quy (recursive)*, *ký tự đại diện (wildcard/glob)*, *idempotent (lặp-an-toàn)*. Đừng lo nếu giờ chưa hiểu — ta sẽ định nghĩa từng cái đúng lúc.

1 Khởi động: vào sân tập (làm trước mọi thứ)

Nguyên tắc số 1 của chương này: **chỉ thực hành trong sân tập `~/linux-lab`, tuyệt đối không thực hành ở thư mục thật** (như Desktop, Documents, hay nơi chứa hình ảnh/giấy tờ của bạn).

Mở **Terminal** (ứng dụng có sẵn trên máy Mac: nhấn `Cmd + Space`, gõ `Terminal`, nhấn `Enter`). Gõ:

```
mkdir -p ~/linux-lab/ch3
cd ~/linux-lab/ch3
pwd
```

Giải thích từng phần:

- `mkdir` = *make directory* (tạo thư mục). `-p` là một **tùy chọn** (option) nghĩa là “tạo cả các thư mục cha nếu chưa có, và đừng báo lỗi nếu nó đã tồn tại”. Nhờ `-p` bạn chạy lại lệnh này nhiều lần cũng không sao.
- `~` là lối viết tắt của thư mục nhà (home) của bạn, ví dụ `/Users/ten-cua-ban`.
- `cd` = đi vào thư mục đó. `pwd` = in ra đường dẫn hiện tại để bạn chắc chắn mình đang đứng đúng chỗ.

Kết quả mong đợi (dòng cuối, `pwd` in ra):

```
/Users/ten-cua-ban/linux-lab/ch3
```

Ghi chú: Máy bạn có thể hiển thị khác

chỗ `ten-cua-ban` sẽ là tên tài khoản trên máy bạn. Đó là điều bình thường, không phải lỗi. Miễn là đường dẫn kết thúc bằng `/linux-lab/ch3` là đúng.

Mẹo: Vì sao gõ `pwd`?

Đây đã là thói quen của người chuyên nghiệp: “*Trước khi tạo/sửa/xóa, hãy chắc chắn mình đang ở đúng thư mục.*” **Nhằm thư mục là nguyên nhân số 1 của tai nạn xóa file** — và `pwd` chính là công cụ cho bạn biết mình đang **đứng ở đâu**.

Lưu ý nhỏ về máy đã cấu hình sẵn

Trên một số máy đã được người dùng tinh chỉnh trước (qua “dotfiles”, `oh-my-zsh`...), các lệnh `rm`, `cp`, `mv` có thể **tự động hỏi xác nhận** do đã được đặt sẵn “bí danh” (alias) kiểu `rm='rm -i'`. Trong sách này, ta mô tả **hành vi mặc định gốc của macOS**. Nếu máy bạn hỏi trong khi sách nói “không hỏi”, đó là do cấu hình riêng — **đừng ý lại vào nó**, vì máy khác (đặc biệt là máy chủ thật ngoài đời) sẽ **KHÔNG** hỏi.

2 XEM nội dung file

Trước khi tạo file thật, ta cần một file để xem. Tạo nhanh một file vài dòng (lệnh `printf` dưới đây bạn cứ gõ y nguyên, ta sẽ học `echo`/`>` kỹ ở Mục 2):

```
printf "Dong 1\nDong 2\nDong 3\nDong 4\nDong 5\nDong 6\n" > tho.txt
```

`\n` nghĩa là “xuống dòng”. File `tho.txt` giờ có 6 dòng.

2.1 cat — in TOÀN BỘ file ra màn hình

`cat` (đọc là “cát”, viết tắt của *concatenate* = nối) đổ toàn bộ nội dung file ra Terminal.

```
cat tho.txt
```

Kết quả mong đợi:

```
Dong 1
Dong 2
Dong 3
Dong 4
Dong 5
Dong 6
```

- **Dừng khi:** file **ngắn** (vài dòng đến vài chục dòng).
- **Đừng dừng khi:** file dài hàng nghìn dòng — nó sẽ trôi vèo qua màn hình, bạn không kịp đọc.

2.2 less — cuộn xem file DÀI một cách thoải mái

`less` mở file trong một “cửa sổ xem” cho phép cuộn lên xuống, không làm rời Terminal.

```
less tho.txt
```

Khi đang ở trong `less`:

Phím	Tác dụng
↓ / ↑ hoặc j / k	cuộn xuống / lên từng dòng
Space	xuống một trang
/tu-khoa rồi Enter	tìm kiếm chữ “tu-khoa”; nhấn n để tới kết quả tiếp theo
q	thoát (quan trọng nhất — nhớ kỹ phím q!)

Trần an

Nhiều người mới hoảng vì “vào `less` rồi không ra được”. Bí mật chỉ là: **nhấn q**. Thử ngay: gõ `less tho.txt`, cuộn vài cái, gõ /Dong rồi Enter (nó nhảy tới chữ “Dong”), cuối cùng nhấn q để về Terminal.

Nếu lỡ gõ nhầm gì đó trong less và màn hình trông lạ (ví dụ đang ở chế độ tìm kiếm, hay vừa bấm nhầm phím): cứ nhấn phím Esc một cái, rồi nhấn q — gần như luôn đưa bạn về lại Terminal an toàn.

- **Dừng khi:** file dài, log dài, hoặc bạn muốn tìm một chữ trong file.

2.3 head — xem vài dòng ĐẦU

```
head -n 3 tho.txt
```

Kết quả mong đợi:

```
Dong 1
Dong 2
Dong 3
```

- `-n 3` nghĩa là “lấy 3 dòng đầu”. Bỏ `-n 3` đi thì mặc định `head` lấy 10 dòng đầu.
- **Dùng khi:** chỉ muốn liếc nhanh phần đầu một file lớn (ví dụ kiểm tra một file dữ liệu có đúng định dạng không).

2.4 tail — xem vài dòng CUỐI

```
tail -n 2 tho.txt
```

Kết quả mong đợi:

```
Dong 5
Dong 6
```

`tail` còn một “siêu năng lực” rất quan trọng sau này: **theo dõi file đang được ghi thêm theo thời gian thực** bằng `tail -f` (chữ `f` = *follow*, theo dõi):

```
# (Chi giới thiệu - chưa cần chạy bay gio)
tail -f /duong-dan/toi/file-log.log
```

Mẹo: Vì sao quan trọng?

Khi bạn làm DevOps, máy chủ liên tục ghi **log** (nhật ký hoạt động) vào file. `tail -f` cho bạn “xem trực tiếp” những dòng mới xuất hiện — như xem camera trực tiếp thay vì xem lại băng. Để thoát `tail -f`, nhấn **Control + C**. Ta sẽ dùng nhiều ở các chương về máy chủ; giờ chỉ cần biết nó tồn tại.

Tự kiểm tra 1

File `tho.txt` có 6 dòng. Lệnh nào in ra đúng 2 dòng cuối? Lệnh nào để **thoát** khỏi `less`? (Đáp án ở cuối chương.)

3 TẠO file

3.1 touch — tạo file RỖNG (hoặc cập nhật thời gian)

```
touch trong.txt
ls -l trong.txt
```

Kết quả mong đợi (đại khái):

```
-rw-r--r--  1 ten-cua-ban  staff  0 Jun 26 10:00 trong.txt
```

- Con số 0 (đứng trước ngày giờ) là **kích thước = 0 byte** → file hoàn toàn rỗng.
- `ls -l` là `ls` ở dạng “danh sách dài” (long), hiện thêm quyền, chủ sở hữu, kích thước, thời gian.

Ghi chú: Máy bạn có thể hiển thị khác

ngày giờ sẽ khác; **cột thứ hai** (tên chủ sở hữu, ví dụ `ten-cua-ban`) và **cột thứ ba** (tên nhóm) cũng tùy máy — nhóm có thể là `staff`, `wheel`, `admin` hoặc tên khác đều bình thường, không phải lỗi. Điều **duy nhất** cần để ý là số 0 (kích thước) và tên `trong.txt`.

Nếu file **đã tồn tại**, `touch` không xóa nội dung — nó chỉ **cập nhật lại thời gian sửa đổi** thành “bây giờ”. (Tính năng này hữu ích về sau, ví dụ ép một công cụ build coi file là “vừa mới thay đổi”).

3.2 echo + > — tạo file VÀ ghi nội dung (GHI ĐỀ)

`echo` đơn thuần là in một dòng chữ ra màn hình:

```
echo "Xin chào file"
```

→ in ra: `Xin chào file`

Dấu `>` gọi là **chuyển hướng** (redirect): thay vì in ra màn hình, nó **đổ chữ đó vào file**:

```
echo "Xin chào file" > ghichu.txt
cat ghichu.txt
```

Kết quả mong đợi:

```
Xin chào file
```


CỰC KỲ QUAN TRỌNG — > luôn GHI ĐỀ

Nếu `ghichu.txt` đã có sẵn nội dung, dấu `>` sẽ **xóa sạch nội dung cũ** và thay bằng nội dung mới, **không hỏi han gì cả**. Đây là một trong những cạm bẫy mất dữ liệu kinh điển nhất của người mới.

3.3 `echo + >>` — NỐI THÊM vào cuối file

Hai dấu `>>` nghĩa là “thêm vào cuối, giữ nguyên nội dung cũ”:

```
echo "Dong thu hai" >> ghichu.txt
cat ghichu.txt
```

Kết quả mong đợi:

```
Xin chào file
Dòng thứ hai
```

Bây giờ hãy **tự mình thấy sự khác biệt** chết người giữa `>` và `>>`. Chạy thử:

```
echo "TÔI GHI ĐỀ" > ghichu.txt
cat ghichu.txt
```

Kết quả mong đợi:

```
TÔI GHI ĐỀ
```

Hai dòng cũ (“Xin chào file” và “Dòng thứ hai”) đã **biến mất vĩnh viễn**. Đó chính là `>`.

Khôi phục lại để tiếp tục lab:

```
echo "Xin chao file" > ghichu.txt
echo "Dong thu hai" >> ghichu.txt
cat ghichu.txt
```

Bảng so sánh `>` và `>>`

Ký hiệu	Tên	Hành vi	Rủi ro
<code>></code>	ghi đè	Xóa hết nội dung cũ, ghi mới	Cao — mất dữ liệu nếu nhầm
<code>>></code>	nối thêm	Giữ cũ, thêm vào cuối	Thấp

Tự kiểm tra 2

Bạn có file `cauhinh.txt` đang chứa 100 dòng câu hình quý giá. Bạn muốn thêm 1 dòng vào cuối. Gõ `echo "dong moi" > cauhinh.txt` hay `echo "dong moi" >> cauhinh.txt`? Vì sao? (Đáp án ở cuối chương.)

Góc Senior #1 — > vs >> và bài học “không hoàn tác”

Trong production, người ta hiếm khi gõ tay `echo > file`. Nhưng `>` xuất hiện *khắp* nơi trong script tự động. Một dòng script viết nhầm `>` thành nơi đáng ra phải là `>>` có thể **xóa trắng file log hoặc file cấu hình** của hàng nghìn máy chủ chỉ trong một nốt nhạc. Quy tắc của người kỳ cựu:

- Khi **muốn thêm** → luôn `>>`. Khi **cố ý thay mới toàn bộ** → mới dùng `>`, và phải tỉnh táo.
- Trước khi chạy script có `>` lên file thật, **backup** (sao lưu) file đó trước. “Không hoàn tác được” là đặc tính cốt lõi của dòng lệnh — bạn không có nút Undo như trong Word.

4 SAO CHÉP file và thư mục — cp

`cp` = *copy*. Cú pháp: `cp <nguồn> <đích>`. Nguồn được giữ nguyên; đích là bản sao mới.

4.1 Sao chép một file

```
cp ghichu.txt ghichu-backup.txt
ls
```

Kết quả mong đợi (`ls` hiện ra, thứ tự có thể khác):

```
ghichu-backup.txt  ghichu.txt  tho.txt  trong.txt
```

Giờ bạn có 2 file giống hệt nhau: bản gốc `ghichu.txt` và bản sao `ghichu-backup.txt`.

4.2 Sao chép cả THƯ MỤC — cần -R

Nếu bạn thử `cp` một thư mục mà không có cờ đặc biệt, macOS sẽ báo lỗi (đại khái) kiểu:

```
cp: hop-tai-lieu is a directory (not copied).
```

Để sao chép thư mục cùng toàn bộ nội dung bên trong, dùng cờ `-R`:

```
mkdir hop-tai-lieu
cp ghichu.txt hop-tai-lieu/
cp -R hop-tai-lieu hop-tai-lieu-backup
```

```
ls
ls hop-tai-lieu-backup
```

Kết quả mong đợi: thư mục `hop-tai-lieu-backup` xuất hiện, và bên trong nó có `ghichu.txt`.

- `-R` = *recursive* (**đệ quy** — đây là lần đầu ta gặp từ này trong bài) = “đi vào thư mục, sao chép luôn mọi thứ bên trong, kể cả thư mục con của thư mục con...”.
- Trên `macOS/BSD`, `-R` (chữ hoa) là cách viết **chuẩn** cho sao chép thư mục. Cờ `-r` (chữ thường) cũng được chấp nhận, nhưng hãy quen dùng `-R` cho đúng chuẩn `macOS`.

4.3 `cp -i` — hỏi trước khi GHI ĐỀ (an toàn)

Mặc định, nếu đích **đã tồn tại**, `cp` sẽ **ghi đè im lặng** — nuốt mất file đích cũ. Cờ `-i` (*interactive*, hỏi tương tác) bắt nó dừng lại hỏi bạn:

```
cp -i tho.txt ghichu.txt
```

Vì `ghichu.txt` đã tồn tại, bạn sẽ thấy câu hỏi:

```
overwrite ghichu.txt? (y/n [n])
```

Gõ `n` rồi `Enter` để **từ chối** (giữ nguyên `ghichu.txt`).

Trần an

gõ `n` là an toàn — không có gì bị thay đổi. Cứ mạnh dạn thử rồi gõ `n`.

Tự kiểm tra 3

Vì sao `cp -i` an toàn hơn `cp` thường khi file đích đã tồn tại? (*Đáp án ở cuối chương.*)

Góc Senior #2 — `cp -a` khi backup, và “copy giữ nguyên hồn vía”

Khi sao lưu thật (không chỉ chép nội dung mà còn muốn giữ **quyền truy cập, thời gian tạo, liên kết tượng trưng...**), Senior dùng `cp -a` (*archive* — lưu trữ). Nó như `cp -R` nhưng **giữ nguyên thuộc tính** của file. Ví dụ sao lưu một thư mục dự án mà vẫn giữ đúng quyền:

```
cp -a hop-tai-lieu hop-tai-lieu-archive
```

Vì sao quan trọng trong production? Một file cấu hình bị đổi quyền sau khi copy có thể khiến dịch vụ **không khởi động được**, hoặc tệ hơn, **để lộ quyền** cho người không đáng. “Sao chép đúng cách” cũng là một kỹ năng an toàn.

5 DI CHUYỂN và ĐỔI TÊN — mv

`mv` = *move*. Một lệnh, hai công dụng:

- **Di chuyển** file/thư mục sang chỗ khác.
- **Đổi tên** file/thư mục (bản chất là “di chuyển sang chính chỗ cũ nhưng với tên mới”).

5.1 Đổi tên

```
mv trong.txt ghi-chu-2.txt
ls
```

Kết quả mong đợi: `trong.txt` biến mất, thay vào đó có `ghi-chu-2.txt`. Nội dung (rõ ràng) vẫn nguyên, chỉ cái tên thay đổi.

5.2 Di chuyển vào thư mục

```
mkdir luu-tru
mv ghichu-backup.txt luu-tru/
ls
ls luu-tru
```

Kết quả mong đợi: `ghichu-backup.txt` không còn ở thư mục hiện tại nữa, mà nằm trong `luu-tru/`.

5.3 `mv -i` — hỏi trước khi GHI ĐÈ

Lưu ý: `mv` ghi đè im lặng

Giống `cp`, `mv` **ghi đè im lặng** nếu đích trùng tên. Khác biệt còn nguy hiểm hơn `cp`: với `cp`, file nguồn vẫn còn; với `mv` ghi đè, bạn vừa **mất file đích cũ**, vừa không còn bản nào khác của nó.

Cờ `-i` cứu bạn:

```
mv -i ghi-chu-2.txt ghichu.txt
```

Sẽ hiện:

```
overwrite ghichu.txt? (y/n [n])
```

Gõ `n` để giữ an toàn (không đổi gì). Nếu thật sự muốn đè thì mới gõ `y`.

Tự kiểm tra 4

Sau `mv a.txt b.txt` (trong đó `b.txt` đã tồn tại từ trước và bạn **KHÔNG** dùng `-i`), điều gì xảy ra với nội dung cũ của `b.txt`? (*Đáp án ở cuối chương.*)

Góc Senior #3 — `mv` giữa hai phân vùng thực ra là “copy + delete”

Khi `mv` trong **cùng một ổ đĩa/phân vùng**, hệ thống chỉ đổi “nhãn địa chỉ” — nhanh tức thì, dù file nặng vài GB. Nhưng khi `mv` **sang ổ khác** (ví dụ từ ổ trong ra ổ USB ngoài), thực chất hệ thống phải **sao chép toàn bộ dữ liệu sang, rồi xóa bản gốc**. Hệ quả thực tế:

- Di chuyển file lớn sang ổ ngoài sẽ **chậm** và **tốn dung lượng tạm thời** ở cả hai nơi.
- Nếu quá trình bị ngắt giữa chừng (rút USB, mất điện), bạn có thể có **file sao chép dở dang**. → Senior thường copy xong, **kiểm tra**, rồi mới xóa nguồn, thay vì tin tưởng tuyệt đối vào `mv` qua ổ ngoài.

6 XÓA file — phần NGUY HIỂM NHẤT, đọc kỹ

CẢNH BÁO LỚN NHẤT CỦA CẢ CHƯƠNG

Trên Terminal, `rm` **KHÔNG** đưa file vào Thùng rác (Trash) như khi bạn kéo file trong Finder. `rm` **xóa thẳng, không hoàn tác được bằng cách thông thường**. Không có nút Undo, không có “khôi phục từ Thùng rác”. Hãy đối xử với `rm` như một con dao sắc: cực kỳ hữu ích, nhưng phải cầm cẩn thận.

6.1 `rm` — xóa một file

Trước khi xóa, **luôn `ls` để nhìn mình sắp xóa gì** — và nếu sắp dùng `rm -r` hay `*`, hãy `pwd` để chắc chắn mình **đang đứng đúng thư mục** (đây là thói quen vàng, ta sẽ nhắc lại nhiều lần):

```
ls
rm tho.txt
ls
```

Kết quả mong đợi: sau lệnh `rm`, `tho.txt` biến mất khỏi danh sách. Vĩnh viễn.

6.2 `rm -i` — hỏi xác nhận từng file (an toàn cho người mới)

```
rm -i ghichu.txt
```

Hiện (lưu ý: prompt của `rm -i` khác với `cp -i`/`mv -i` — nó **không** có gợi ý `[n]`):

```
remove ghichu.txt?
```

Gõ y rồi **Enter** để xóa, hoặc **n** để giữ lại.

Trần an quan trọng

Với `rm -i`, nếu bạn **chỉ nhấn Enter** mà không gõ gì, file sẽ **KHÔNG** bị xóa (mặc định coi như “không”). Tức là khi phân vân, cứ nhấn **Enter** là an toàn. Với người mới, **tập thói quen luôn dùng `rm -i`** cho tới khi bạn thật vững.

6.3 `rmdir` — chỉ xóa thư mục RỖNG (an toàn)

`rmdir` (*remove directory*) chỉ xóa được thư mục **không còn gì bên trong**. Đây là một “lưới an toàn” tự nhiên: nếu thư mục còn file, nó sẽ từ chối, nhắc bạn rằng “ê, trong này còn đồ đấy”.

```
mkdir thu-muc-rong
rmdir thu-muc-rong
ls
```

Kết quả mong đợi: `thu-muc-rong` biến mất, không báo lỗi.
Thử xóa một thư mục **còn nội dung** bằng `rmdir`:

```
rmdir luu-tru
```

Kết quả mong đợi (báo lỗi — đây là điều **TỐT**):

```
rmdir: luu-tru: Directory not empty
```

Trần an

dòng “Directory not empty” **không phải lỗi của bạn** — đó là `rmdir` đang **bảo vệ bạn**, không cho lỡ tay xóa thư mục còn dữ liệu.

6.4 `rm -r` — xóa thư mục VÀ toàn bộ nội dung (đệ quy)

Khi bạn **thật sự** muốn xóa cả thư mục lẫn mọi thứ bên trong, dùng `rm -r` (`-r` = recursive, **đệ quy** — đi vào và xóa hết mọi thứ bên trong, kể cả thư mục con):

```
pwd
ls luu-tru
rm -r luu-tru
ls
```

Kết quả mong đợi: cả `luu-tru` lẫn `ghichu-backup.txt` bên trong nó đều biến mất.

Quy tắc vàng nâng cấp cho `rm -r`

`rm -r` mạnh và **không hỏi**. Nó cuốn sạch cả cây thư mục. Vì vậy:

`pwd` trước (mình ĐANG Ở ĐÂU?) → `ls` (mình SẮP XÓA GÌ?) → rồi mới `rm -r`.

Vì sao cần cả `pwd`? Vì rủi ro lớn nhất của `rm -r` không phải là “xóa nhầm file trong thư mục đúng”, mà là **bạn đang đứng nhầm thư mục mà không biết**. `ls` chỉ cho thấy *nội dung* — nếu bạn đứng nhầm chỗ, `ls` vẫn hiện ra một danh sách trông “hợp lý” và bạn vẫn xóa nhầm. Chỉ có `pwd` mới xác nhận **vị trí**.

Mẹo an toàn cho người mới — `rm -ri`

Bạn có thể **ghép** `-r` với `-i` thành `rm -ri` *thumuc*. Lệnh này xóa đệ quy cả cây thư mục **nhưng hỏi xác nhận từng mục một**. Khi bạn cần xóa một thư mục mà chưa thật chắc chắn bên trong có gì, `rm -ri` là cách thận trọng: bạn nhìn từng cái rồi mới gật đầu. (`-r` và `-i` **không** loại trừ nhau.)

Tự kiểm tra 5

Bạn muốn xóa thư mục `tam` còn chứa 3 file bên trong. `rmdir tam` hay `rm -r tam`? Và bạn nên gõ những lệnh nào *trước* để nhìn cho chắc? (*Đáp án ở cuối chương.*)

6.5 `rm -rf` và những thảm họa kinh điển — ĐỌC THẬT KỸ

`rm -rf` ghép hai cờ:

- `-r` = đệ quy (xóa cả cây thư mục).
- `-f` = *force* (ép buộc) = “đừng hỏi gì hết, đừng báo lỗi, cứ xóa”.

Tức là `rm -rf` = “**xóa sạch mọi thứ, không hỏi, không cảnh báo**”. Đây là lệnh phá hủy mạnh nhất bạn gặp trong chương này.

TUYỆT ĐỐI GHI NHỚ — những lệnh KHÔNG BAO GIỜ được gõ liều

- `rm -rf /` → lệnh phá hủy có chủ đích nhắm vào **gốc hệ thống /**. *Lưu ý chính xác về macOS*: macOS hiện đại có cơ chế bảo vệ tên là **SIP (System Integrity Protection)**, và bản thân `rm` cũng có “phanh” từ chối xóa thẳng /, nên trên thực tế **một phần lớn hệ thống được bảo vệ và lệnh này có thể không xóa hết được**. Nhưng đừng vì thế mà coi thường: nó **vẫn có thể phá hỏng dữ liệu cá nhân của bạn và làm hỏng hệ điều hành**. Tuyệt đối không bao giờ thử.
- `rm -rf ~` → xóa **toàn bộ thư mục nhà** của bạn: tài liệu, hình ảnh, mọi thứ bạn tạo ra. Phần này **không** được SIP bảo vệ → mất là mất thật.
- `rm -rf *` khi đang đứng nhầm thư mục → xóa sạch mọi thứ trong thư mục

đó.

Trên macOS, những thứ bị xóa kiểu này **không vào Thùng rác để bạn lôi ra lại**. Mất là mất.

Quy tắc sống còn:

1. **Không bao giờ** gõ `rm -rf` với đường dẫn bắt đầu bằng `/` hoặc `~` trừ khi bạn đã suy nghĩ rất kỹ và 100% chắc chắn.
2. Luôn `pwd` rồi `ls` trước để biết chính xác mình đang ở đâu và sắp xóa gì.
3. **Gõ chậm**, đặc biệt khi có `rm`, có `-rf`, hoặc có dấu `*`.
4. Khi học, dùng `rm -i` (hoặc `rm -ri` cho thư mục) thay cho `rm -f`. `-i` hỏi để cứu bạn; `-f` thì không.

7 KÝ TỰ ĐẠI DIỆN (wildcard / glob) — sơ lược nhưng phải biết cảnh giác

Ký tự đại diện (*wildcard*, hay *glob*) là cách nói “khớp với nhiều file một lúc”. Phổ biến nhất là dấu `*` = “khớp với bất kỳ chuỗi ký tự nào”.

Tạo vài file để thử (an toàn, trong sân tập):

```
touch a.txt b.txt c.txt ghichu.log
ls *.txt
```

Kết quả mong đợi: chỉ liệt kê các file kết thúc bằng `.txt`:

```
a.txt  b.txt  c.txt
```

`*.txt` nghĩa là “mọi file có tên kết thúc bằng `.txt`”. File `ghichu.log` không khớp nên không hiện.

Một điểm quan trọng cần biết về `*`

dấu `*` **không khớp các file ẩn** (file có tên bắt đầu bằng dấu chấm, ví dụ `.bashrc`). Vì vậy `rm *` xóa mọi file **thường** nhưng **bỏ qua file ẩn**. Đừng dựa vào điều này để “dọn sạch” thư mục — nó chỉ là một chi tiết kỹ thuật, không phải lưỡi an toàn.

Cạm bẫy `*` + `rm` — hai rủi ro tách bạch

Rủi ro 1 — bản thân `rm *` đã đủ nguy hiểm. Lệnh `rm *.txt` xóa **MỌI** file `.txt` trong thư mục hiện tại cùng một lúc; còn `rm *` (chỉ một dấu sao) xóa **MỌI file thường** trong thư mục. Đây là rủi ro **chính** của wildcard: một lệnh ngán tác động lên rất nhiều file, và **không hoàn tác được**. Bạn không cần gõ sai gì cả —

chỉ cần đứng nhầm thư mục là đủ thành thảm họa.

Rủi ro 2 — gõ thừa dấu cách. Một dấu cách thừa biến `rm *.txt` thành `rm *.txt`. Khi đó shell mở rộng `*` thành **TẤT CẢ file trong thư mục TRƯỚC**, xóa sạch chúng; phần `.txt` phía sau chỉ được hiểu là **một tên file riêng** tên đúng là `.txt` (thường không tồn tại). Hậu quả: `rm` sẽ in thêm một dòng lỗi kiểu `rm: .txt: No such file or directory` — **nhưng đừng nhầm dòng lỗi đó là “lệnh không chạy gì”**: thực tế `*` đã kịp xóa sạch mọi file *trước khi* dòng lỗi xuất hiện. (Cùng cơ chế: một dấu cách lạc kiểu `rm -rf / tmp` thay vì `rm -rf /tmp` là cạm bẫy chết người riêng — luôn soi kỹ dấu cách quanh `/`.)

Vì vậy, quy tắc tuyệt đối: trước khi `rm` với `*`, hãy chạy `pwd` (đúng thư mục chưa?) rồi `ls` với **cùng mẫu** đó để xem danh sách:

```
pwd          # mình có đang đứng đúng thư mục không?
ls *.txt     # xem trước: mình sắp xóa đúng những file này chứ?
rm *.txt     # giờ mới xóa, vì đã nhìn tan mắt
```

Chạy thử an toàn trong lab:

```
pwd
ls *.txt
rm *.txt
ls
```

Kết quả mong đợi: `a.txt`, `b.txt`, `c.txt` biến mất; `ghichu.log` vẫn còn (vì không khớp `*.txt`).

Tự kiểm tra 6

`ls *.txt` cho thấy 3 file. Bạn yên tâm gõ `rm *.txt`. Vì sao bước `ls` (và `pwd`) trước đó lại quan trọng đến vậy? (*Đáp án ở cuối chương.*)

Góc Senior #4 — quoting, --, idempotent, alias an toàn, và vì sao “biến rộng” giết cả hệ thống

Đây là phần “tư duy người kỳ cựu” — đọc để hiểu **vì sao**, chưa cần thuộc lòng.

(a) Thảm họa biến rộng — bài học triệu đô. Trong script, người ta hay viết:

```
rm -rf "$DIR"/
```

Ý định: xóa nội dung thư mục lưu trong biến `DIR`. **Nhưng nếu vì lỗi nào đó `DIR` bị rỗng** (chưa gán giá trị), dòng trên trở thành:

```
rm -rf /
```

→ nhắm vào gốc hệ thống. Đây là loại bug đã từng xóa sạch sản phẩm của nhiều công ty thật (trên các hệ Linux không có SIP, hậu quả là toàn bộ máy chủ). Bài

học: luôn kiểm tra biến trước khi đưa vào `rm`, và **không tin tưởng mù quáng vào biến**.

(b) **Quoting (đặt trong dấu nháy)**. Tên file có dấu cách như `bao cao.txt` mà không bọc nháy sẽ bị hiểu thành **hai** đối số `bao` và `cao.txt`. Senior luôn bọc: `rm "bao cao.txt"`.

(c) **Dấu `--` để kết thúc danh sách tùy chọn**. Một file lỗi tên là `-rf` sẽ bị `rm` hiểu nhầm thành cờ. `rm -- -rf` báo cho `rm`: “hết tùy chọn rồi, phần sau là TÊN FILE”. Mẹo nhỏ nhưng cứu nguy thật.

(d) **Idempotent (tính “lặp-an-toàn”)**. Một thao tác *idempotent* là thao tác **chạy 1 lần hay 10 lần đều đưa hệ thống về cùng một trạng thái cuối** — chạy lại không gây thêm tác dụng phụ và không báo lỗi. (Lưu ý: đây **không** phải “bất biến/không thay đổi gì” — lần chạy đầu vẫn thay đổi trạng thái; cái “an toàn” là ở những lần lặp sau.) Ví dụ: `mkdir -p ~/linux-lab/ch3` là idempotent (chạy lại không lỗi, thư mục vẫn ở đó); còn `rm` thì không (chạy lần 2 báo “không tìm thấy file”). DevOps quý tính idempotent vì script tự động hay chạy lặp, và ta cần kết quả **dự đoán được**.

(e) **Lưới an toàn không phụ thuộc trí nhớ — alias và “Trash”**. Vì con người hay quên gõ `-i` đúng lúc nguy hiểm nhất, nhiều người đặt sẵn một “bí danh” (alias) trong file cấu hình shell để **`rm` luôn hỏi**:

```
alias rm='rm -i'
```

Ngoài ra còn có những công cụ “đưa file vào Thùng rác thay vì xóa thẳng” (ví dụ lệnh `trash` cài thêm) để bạn còn đường khôi phục. **Nhưng nhớ kỹ**: alias chỉ tồn tại trên *máy của bạn*. Khi sang máy chủ thật hoặc máy đồng nghiệp, `rm` lại “trần trụi” và **không hỏi**. → Hãy coi alias/Trash là *lưới an toàn phụ*, còn thói quen `pwd + ls + gõ chậm` mới là *kỹ năng gốc*.

(f) **Vì sao “backup trước thao tác phá hủy”**. Senior luôn tự hỏi: “*Nếu lệnh này sai, mình mất gì? Khôi phục được không?*” Nếu câu trả lời là “mất nhiều, khó khôi phục” → **backup trước**, rồi mới làm.

8 LAB CHÍNH — gõ tay toàn bộ, an toàn 100% (chỉ trong `~/linux-lab/ch3`)

Thực hành ngay — Lab chính

Đây là bài thực hành “chính thức” gói gọn tất cả những gì đã học. Hãy gõ **từng dòng**, đọc kết quả, rồi mới sang dòng tiếp theo. (Nếu trước đó bạn đã lỡ tạo/xóa lung tung, không sao — bước 1 dựng lại sạch sẽ.)

Bước 1 — Tạo & vào sân tập:

```
mkdir -p ~/linux-lab/ch3
cd ~/linux-lab/ch3
```

```
pwd
```

Mong đợi: pwd in ra đường dẫn kết thúc bằng /linux-lab/ch3.

Bước 2 — Tạo file có nội dung:

```
echo "Xin chao file" > ghichu.txt
cat ghichu.txt
```

Mong đợi:

```
Xin chào file
```

Bước 3 — Nối thêm, rồi so sánh với ghi đè:

```
echo "Dong thu hai" >> ghichu.txt
cat ghichu.txt
```

Mong đợi (2 dòng):

```
Xin chào file
Dòng thứ hai
```

(Ghi nhớ: nếu lúc này bạn dùng > thay vì >>, dòng đầu sẽ bị xóa mất. Đó là khác biệt sống còn.)

Bước 4 — Tạo file rỗng và soi kích thước:

```
touch trong.txt
ls -l
```

Mong đợi: trong danh sách, trong.txt có kích thước 0.

Bước 5 — Sao chép:

```
cp ghichu.txt ghichu-backup.txt
ls
```

Mong đợi: thấy cả ghichu.txt và ghichu-backup.txt.

Bước 6 — Đổi tên & di chuyển:

```
mv trong.txt ghi-chu-2.txt
mkdir luu-tru
mv ghichu-backup.txt luu-tru/
ls
ls luu-tru
```

Mong đợi: trong.txt thành ghi-chu-2.txt; ghichu-backup.txt nằm trong luu-tru/, không còn ở thư mục hiện tại.

Bước 7 — Xóa an toàn có hỏi, rồi “ls trước rm sau”:

```
rm -i ghi-chu-2.txt
```

Khi hỏi `remove ghi-chu-2.txt`? → gõ y rồi **Enter**. (Nhớ: nếu chỉ nhấn **Enter** không gõ gì, file sẽ **không** bị xóa.)

```
ls
rm ghichu.txt
ls
```

Mong đợi: `ghi-chu-2.txt` và `ghichu.txt` lần lượt biến mất.

Bước 8 — Dọn toàn bộ sân tập ch3 (dùng `rm -r` CÓ CHỦ ĐÍCH):

```
cd ~
pwd
rm -r ~/linux-lab/ch3
ls ~/linux-lab
```

Mong đợi: `pwd` in ra thư mục nhà của bạn (kết thúc bằng tên tài khoản, ví dụ `/Users/ten-cua-ban`); sau `rm -r`, thư mục `ch3` biến mất khỏi `~/linux-lab`.

Vì sao ở đây ta được phép dùng `rm -r`?

Vì hội tụ đủ điều kiện an toàn:

1. Đường dẫn **viết rõ ràng, tường minh** (`~/linux-lab/ch3`) — không dùng biến chưa rõ, không dùng `*` mơ hồ.
2. Đây là **thư mục tập của chính bạn**, nội dung chỉ là file thực hành, mất cũng không sao.
3. Bạn đã `cd ~` rồi `pwd` để **xác nhận** mình không đứng bên trong thư mục sắp xóa, và biết rõ mình đang ở đâu.

Đây chính là cách Senior dùng `rm -r`: có chủ đích, đường dẫn rõ ràng (không phải `./` mơ hồ), đã `pwd` xác nhận vị trí, đã suy nghĩ trước. Không bao giờ là “gõ cho nhanh”.

9 HIỂU LẦM THƯỜNG GẶP (đọc để khỏi “dính bẫy”)

Hiểu lầm (SAI)	Sự thật (ĐÚNG)
“ <code>rm</code> đưa file vào Thùng rác như Finder.”	SAI. <code>rm</code> xóa thẳng, không qua Thùng rác, không hoàn tác bằng cách thông thường.
“ <code>></code> và <code>>></code> na ná nhau.”	SAI. <code>></code> ghi đè (xóa nội dung cũ); <code>>></code> nối thêm . Nhầm là mất dữ liệu.

Hiểu lầm (SAI)	Sự thật (ĐÚNG)
“Xóa nhầm thì khôi phục dễ mà.”	Không nên trông cậy. Có công cụ khôi phục chuyên sâu nhưng phức tạp, tốn kém, không chắc thành công. Cách an toàn duy nhất là đừng xóa nhầm + backup .
“cp/mv sẽ hỏi trước khi đè.”	SAI theo mặc định — chúng đè im lặng . Muốn được hỏi phải thêm -i . (Trừ khi máy bạn đã cài alias sẵn — đừng ý lại.)
“rmkdir xóa được mọi thư mục.”	SAI. rmdir chỉ xóa thư mục rỗng . Thư mục còn nội dung cần rm -r (cẩn thận).
“-r và -i loại trừ nhau.”	SAI. Ghép được: rm -ri thumuc xóa đệ quy nhưng hỏi từng mục — rất hợp cho người mới.
“ls trước là đủ an toàn để rm -r.”	Chưa đủ. ls cho biết <i>nội dung</i> , không cho biết <i>vị trí</i> . Phải pwd để chắc mình không đứng nhầm thư mục .
“rm * xóa sạch MỌI thứ kể cả file ẩn.”	SAI. * không khớp file ẩn (bắt đầu bằng dấu chấm). rm * xóa file thường, bỏ qua file ẩn — nhưng đừng dựa vào đó.
“rm -rf / chắc chắn hủy diệt cả máy Mac.”	Không chính xác. macOS có SIP bảo vệ phần lớn hệ thống nên có thể không xóa hết — nhưng nó vẫn phá dữ liệu và hỏng hệ thống . Tuyệt đối không thử.
“-r và -R trong cp khác nhau lớn trên macOS.”	Cả hai đều sao chép đệ quy; -R là chuẩn BSD/macOS , cứ quen dùng -R .

10 THẺ GHI NHỚ (flashcards)

Che cột phải, tự trả lời, rồi mới nhìn.

Mặt trước (hỏi)	Mặt sau (đáp)
Xem toàn bộ file ngắn	<code>cat file</code>
Cuộn xem file dài; thoát ra	<code>less file</code> ; nhấn q (kẹt thì Esc rồi q)
Tìm chữ trong less	/tu-khoa rồi Enter , n để tới tiếp
5 dòng đầu / 5 dòng cuối	<code>head -n 5 file</code> / <code>tail -n 5 file</code>
Theo dõi log đang ghi	<code>tail -f file</code> (thoát: Control + C)
Tạo file rỗng	<code>touch file</code>
Ghi đè nội dung file	<code>echo "..."</code> > file
Nối thêm vào cuối file	<code>echo "..."</code> >> file
Sao chép file	<code>cp nguồn đích</code>
Sao chép thư mục	<code>cp -R nguồn đích</code>

Mặt trước (hỏi)	Mặt sau (đáp)
Hỏi trước khi đề khi copy	<code>cp -i ...</code>
Di chuyển / đổi tên	<code>mv nguồn đích</code>
Hỏi trước khi đề khi di chuyển	<code>mv -i ...</code>
Xóa file (KHÔNG vào Thùng rác)	<code>rm file</code>
Xóa có hỏi từng file	<code>rm -i file</code> (chỉ Enter = không xóa)
Xóa thư mục rỗng (an toàn)	<code>rmdir thumuc</code>
Xóa thư mục + nội dung	<code>rm -r thumuc</code> (cẩn thận)
Xóa thư mục NHƯNG hỏi từng mục	<code>rm -ri thumuc</code>
Lệnh phá hủy mạnh, dễ gây thảm họa	<code>rm -rf ...</code> (tránh; không bao giờ <code>rm -rf /</code> hay <code>rm -rf ~</code>)
Thói quen vàng trước khi xóa thường	<code>ls</code> trước, <code>rm</code> sau
Thói quen vàng trước <code>rm -r / *</code> * nghĩa là gì	<code>pwd</code> (ở đâu?) → <code>ls</code> (xóa gì?) → rồi mới <code>rm</code> “khớp bất kỳ chuỗi ký tự” (vd <code>*.txt</code>); không khớp file ẩn

11 TÓM TẮT 1 TRANG

- **XEM:** `cat` (file ngắn) · `less` (file dài, / tìm, q thoát, kệt thì Esc rồi q) · `head -n N` (đầu) · `tail -n N` (cuối) · `tail -f` (theo dõi log, thoát Control+C).
- **TẠO:** `touch f` (file rỗng / cập nhật giờ) · `echo "x" > f` (**ghi đề**) · `echo "x" >> f` (**nối thêm**). Nhớ: > xóa nội dung cũ, >> giữ lại.
- **SAO CHÉP:** `cp nguồn đích` · thư mục: `cp -R nguồn đích` · hỏi trước khi đề: `cp -i` · backup giữ thuộc tính: `cp -a`.
- **DI CHUYỂN/ĐỔI TÊN:** `mv nguồn đích` (đề **im lặng** nếu không có `-i`) · an toàn: `mv -i`.
- **XÓA:** `rm f` (**không** vào Thùng rác, **không** hoàn tác) · `rm -i` (hỏi; chỉ Enter = không xóa) · `rmdir` (chỉ thư mục **rỗng**) · `rm -r` (thư mục + nội dung) · `rm -ri` (đệ quy nhưng hỏi từng mục). **`rm -rf` cực nguy hiểm; KHÔNG BAO GIỜ `rm -rf /` hay `rm -rf ~`.**
- **WILDCARD *:** `ls *.txt` xem trước; `rm *.txt` xóa hàng loạt; * **không** khớp file ẩn → **luôn `pwd` + `ls` trước khi `rm`.**
- **THÓI QUEN AN TOÀN:** chỉ tập trong `~/linux-lab`; `pwd` để biết mình ở đâu + `ls` để biết mình sắp xóa gì; `-i` khi `cp/mv/rm` (và `-ri` cho thư mục); gõ chậm khi có `rm`, `-rf`, *; backup trước thao tác phá hủy; alias `rm='rm -i'` là lưới phụ, không thay được thói quen.

12 BÀI TẬP MỞ RỘNG (stretch) — CÓ đáp án

Làm trong sân tập sạch. Dừng lại trước:

```
mkdir -p ~/linux-lab/ch3-stretch
cd ~/linux-lab/ch3-stretch
pwd
```

Bài S1. Tạo file `nhatky.txt` gồm đúng 3 dòng: Sáng, Trưa, Tối — chỉ dùng `echo` với `>` và `>>`. Sau đó in toàn bộ ra.

Bài S2. Tạo 4 file: `note1.txt`, `note2.txt`, `note3.txt`, `anh.png`. Dùng **một** lệnh `ls` (có `*`) để chỉ liệt kê các file `.txt`. Rồi dùng **một** lệnh để xóa **chỉ** các file `.txt`, nhưng **xem trước bằng `pwd` và `ls`**.

Bài S3. Tạo thư mục `du-an` chứa file `readme.txt`. Thử xóa `du-an` bằng `rmdir` và quan sát. Rồi xóa thành công bằng cách đúng.

Bài S4. Tạo `goc.txt` chứa dòng bản gốc. Sao chép thành `sao.txt`. Dùng `cp -i` để thử ghi đè `sao.txt` bằng `goc.txt` — nhưng **từ chối** ghi đè.

Bài S5 (tư duy + thực hành an toàn). Bạn muốn dọn sạch thư mục tập `ch3-stretch` để bắt đầu lại. Hãy làm theo cách **an toàn nhất, tường minh nhất**: xóa cả thư mục bằng **đường dẫn đầy đủ** rồi tạo lại — **không dùng `*`**. Giải thích vì sao cách này an toàn hơn việc gõ một lệnh có dấu `*`.

ĐÁP ÁN / KẾT QUẢ MONG ĐỢI — Bài tập mở rộng

S1.

```
echo "Sang" > nhatky.txt
echo "Trua" >> nhatky.txt
echo "Toi" >> nhatky.txt
cat nhatky.txt
```

Kết quả:

```
Sáng
Trưa
Tối
```

Lưu ý: chỉ dòng đầu dùng `>` (tạo mới/ghi đè), hai dòng sau **phải** `>>` để không xóa dòng trước.

S2.

```
touch note1.txt note2.txt note3.txt anh.png
ls *.txt
```

Kết quả `ls *.txt`:

```
note1.txt  note2.txt  note3.txt
```

Xem trước (cả vị trí lẫn nội dung) rồi mới xóa:

```
pwd
ls *.txt
rm *.txt
ls
```

Sau cùng `ls` chỉ còn `anh.png` (vì `.png` không khớp `*.txt`).

S3.

```
mkdir du-an
touch du-an/readme.txt
rmdir du-an
```

`rmdir` báo lỗi (đây là điều đúng — nó bảo vệ bạn):

```
rmdir: du-an: Directory not empty
```

Xóa đúng cách (đã nhìn trước):

```
ls du-an
rm -r du-an
ls
```

`du-an` biến mất. (Nếu muốn thận trọng hơn nữa, có thể dùng `rm -ri du-an` để được hỏi xác nhận từng mục.)

S4.

```
echo "ban goc" > goc.txt
cp goc.txt sao.txt
cp -i goc.txt sao.txt
```

Khi hiện `overwrite sao.txt? (y/n [n])` → gõ `n` rồi `Enter`. `sao.txt` giữ nguyên (không bị đè). Kiểm chứng cat `sao.txt` vẫn ra bản gốc.

S5.

```
cd ~
pwd
rm -r ~/linux-lab/ch3-stretch
mkdir ~/linux-lab/ch3-stretch
ls ~/linux-lab
```

Giải thích: cách này xóa **chính thư mục `ch3-stretch`** bằng **đường dẫn tuyệt đối, tường minh** (`~/linux-lab/ch3-stretch`) rồi tạo lại thư mục rỗng — kết quả y hệt “dọn sạch nội dung” nhưng **an toàn hơn nhiều** so với lệnh có dấu `*`. Vì sao?

- Đường dẫn ghi rõ ràng, **không phụ thuộc vào việc bạn đang đứng ở đâu** — nên dù bạn quên `cd`, lệnh vẫn nhắm đúng chỗ. (Lệnh kiểu `rm -r ./*` thì ngược lại: nó phụ thuộc hoàn toàn vào thư mục hiện tại, nên nếu bạn lỡ đứng ở Home hay một thư mục thật, nó sẽ quét sạch nội dung chỗ đó.)

- Trước khi xóa, ta `cd ~` rồi `pwd` để **xác nhận** mình không đứng bên trong thư mục sắp xóa và biết rõ vị trí.
- Không có `*` → không có rủi ro “mở rộng ngoài ý muốn” hay “dấu cách thừa”.

(Lưu ý cho người mới: hãy tránh thói quen gõ `rm -r ./*` — nó là đúng cái combo `* + -r` mà cả chương này khuyên cảnh giác. Khi cần “làm lại từ đầu”, cách “xóa cả thư mục bằng đường dẫn đầy đủ rồi `mkdir` lại” vừa đơn giản vừa khó sai hơn.)

ĐÁP ÁN — phần “Tự kiểm tra” trong bài

Tự kiểm tra 1. In 2 dòng cuối: `tail -n 2 tho.txt`. Thoát less: nhấn phím `q` (nếu màn hình trông lạ, nhấn `Esc` rồi `q`).

Tự kiểm tra 2. Dùng `echo "dong moi" >> cauhinh.txt` (hai dấu `>>`, **nổi thêm**). Nếu dùng một dấu `>` sẽ **ghi đè**, xóa sạch 100 dòng cũ — mất dữ liệu.

Tự kiểm tra 3. Vì `cp` thường **ghi đè im lặng** file đích đã tồn tại; còn `cp -i` **dừng lại hỏi overwrite ...?**, cho bạn cơ hội gõ `n` để hủy trước khi mất file đích cũ.

Tự kiểm tra 4. Nội dung cũ của `b.txt` bị **ghi đè và mất hoàn toàn**, không hỏi, không hoàn tác. (Đó là lý do nên dùng `mv -i`.)

Tự kiểm tra 5. `tam` còn file bên trong nên `rmdir` sẽ báo lỗi “Directory not empty”; phải dùng `rm -r tam` (hoặc thận trọng hơn: `rm -ri tam`). Nên gõ `pwd` (chắc chắn đang ở đúng thư mục) và `ls tam` (nhìn rõ sắp xóa gì) **trước** khi xóa.

Tự kiểm tra 6. Vì `rm *.txt` tác động lên **nhiều file cùng lúc** và **không hoàn tác**. Chạy `ls *.txt` với **cùng mẫu** cho bạn thấy **chính xác** danh sách sẽ bị xóa; còn `pwd` đảm bảo bạn không **đứng nhầm thư mục** — vì nếu đứng nhầm chỗ, danh sách `ls` vẫn trông “hợp lý” nhưng bạn lại xóa nhầm dữ liệu thật.

13 KẾ HOẠCH ÔN NGẮT QUẢNG (spaced repetition)

Trí nhớ bền đến từ ôn đúng nhịp, không phải học dồn một lần.

- **Sau 1 ngày:** Không nhìn sách, mở Terminal, tạo `~/linux-lab/ch3-on1`, rồi tự làm lại **Lab Chính bước 2→7** từ trí nhớ. Đặc biệt đọc to khác biệt `>` vs `>>`. Kẹt chỗ nào, đánh dấu chỗ đó.
- **Sau 3 ngày:** Làm lại **Bài tập mở rộng S1→S4** không nhìn đáp án. Tự đặt và trả lời: “`rm` có vào Thùng rác không? `rmdir` xóa được thư mục có file không? `cp` thư mục cần chờ gì? Trước `rm -r` cần gõ gì để biết mình ở đâu?”
- **Sau 1 tuần:** Lấy bộ **Thẻ ghi nhớ** (Mục 9), che cột phải, đọc lướt toàn bộ trong 5 phút. Sau đó nhắm mắt đọc to **3 thói quen an toàn** quan trọng nhất: (1) `pwd + ls` trước `rm -r/*`; (2) dùng `-i` (hoặc `-ri`) khi `cp/mv/rm`; (3) không bao giờ `rm -rf /` hay `rm -rf ~`. Cuối cùng dọn sạch các thư mục ôn tập **theo đúng thói quen an toàn** (xác nhận vị trí trước, dùng đường dẫn tường minh):

```
cd ~
pwd
ls ~/linux-lab
rm -r ~/linux-lab/ch3-on1
```

Lời nhắn cuối chương

Bạn vừa học những lệnh quyền năng nhất — và cũng nguy hiểm nhất — của dòng lệnh. Điều phân biệt người mới với Senior **không phải** là gõ nhanh hơn, mà là **gõ cẩn thận hơn**: luôn biết mình đang ở đâu (`pwd`), luôn nhìn trước khi xóa (`ls`), và luôn tự hỏi “nếu sai thì mình mất gì?”. Giữ vững những thói quen này, bạn đã bước một chân vào tư duy của dân chuyên nghiệp. Hẹn gặp ở Chương 4.